# AWS VPC PEERING PROJECT

Project Report and Validation Evidence

Amazon Web Services (AWS) • VPC • Subnets • EC2 • VPC Peering • Route Tables

| Item | Detail |
|---|---|
| Prepared for | Lavanya Chakkilam |
| Environment | AWS Management Console, AWS CloudShell and SSH from a local terminal |
| AWS Region | Asia Pacific (Sydney) — ap-southeast-2 |
| Evidence date | 23–24 October 2026 |
| Report date | 24 October 2026 |

## 1. Executive Summary

This project demonstrates private network connectivity between two Amazon Virtual Private Clouds (VPCs) using VPC peering. The environment contains VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16), three subnets, three EC2 instances, route-table entries targeting the peering connection, and an active peering connection (pcx-02a215ed0a98562de). AWS CloudShell output confirms all three instances are running and the peering connection is active. Terminal screenshots show SSH access to the bastion host and then to ewb-app-b, a successful ping from App-B (10.20.1.10) to App-A (10.10.2.10), and a successful reverse ping from App-A to App-B, each with 4 packets sent, 4 received and 0% packet loss. The EC2 console shows that ewb-app-a has a public IPv4 address, which differs from the target design (see Section 7). The baseline and failure-and-recovery tests were not captured, and cleanup is not yet evidenced (see Section 7).

## 2. Objectives

- Create two VPCs with non-overlapping IPv4 CIDR ranges.
- Create public and private subnets and associate them with route tables.
- Launch EC2 instances for the two networks plus a bastion host.
- Establish and accept a VPC peering connection between VPC A and VPC B.
- Configure routes so each VPC can send traffic to the other through the peering connection.
- Verify resource states and private connectivity, and capture evidence for this report.

## 3. Architecture and Address Plan

The design uses separate, non-overlapping address spaces. VPC peering is a private network path; it does not create routes automatically and does not bypass security groups or network ACLs, so route tables and security rules must both allow the intended traffic.

| Resource | IPv4 CIDR / address | Purpose / evidence |
|---|---|---|
| VPC A | 10.10.0.0/16 | Workload network A (ewb-vpc-a) |
| VPC A public subnet | 10.10.1.0/24 | ewb-a-public; route table ewb-rt-a-public (Figure 2) |
| VPC A private subnet | 10.10.2.0/24 | ewb-a-private; route table ewb-rt-a-private (Figure 3) |
| VPC B | 10.20.0.0/16 | Workload network B (ewb-vpc-b) |
| VPC B private subnet | 10.20.1.0/24 | ewb-b-private; route table ewb-rt-b-private (Figure 4) |
| EC2: ewb-bastion | 10.10.1.10 | Running (Figure 6) |
| EC2: ewb-app-a | 10.10.2.10 | Running (Figure 6) |
| EC2: ewb-app-b | 10.20.1.10 | Running (Figure 6) |
| VPC peering | pcx-02a215ed0a98562de | Active; 10.10.0.0/16 ↔ 10.20.0.0/16 (Figure 6) |

## 4. Implementation Summary

1. **Create VPCs:** VPC A with 10.10.0.0/16 and VPC B with 10.20.0.0/16. The CIDRs do not overlap, which standard VPC peering requires.
2. **Create subnets:** public subnet 10.10.1.0/24 and private subnet 10.10.2.0/24 in VPC A; private subnet 10.20.1.0/24 in VPC B.
3. **Create VPC peering:** send the peering request between VPC A and VPC B and accept it. The connection status is active.
4. **Configure routing:** associate each subnet with its route table and add the remote VPC CIDR with the peering connection as target.
5. **Launch EC2 instances:** ewb-bastion (10.10.1.10), ewb-app-a (10.10.2.10) and ewb-app-b (10.20.1.10).
6. **Validate:** check subnet, instance, peering and route state in the console and CloudShell, then test SSH and ping over private IPs in both directions.

## 5. Validation Results

| Check | Observed result | Assessment |
|---|---|---|
| VPC CIDRs | 10.10.0.0/16 (ewb-vpc-a) and 10.20.0.0/16 (ewb-vpc-b) shown in the VPC list (Figure 1), the peering page (Figure 5) and the CLI output (Figure 6) | Pass — non-overlapping |
| Subnet inventory | Three subnets, all Available, with the expected CIDRs (Figures 2–4) | Pass |
| Subnet–route table association | ewb-a-public → ewb-rt-a-public; ewb-a-private → ewb-rt-a-private; ewb-b-private → ewb-rt-b-private (Figures 2–4 and 8–11) | Pass |

| Check | Observed result | Assessment |
|---|---|---|
| EC2 instance inventory | ewb-app-b, ewb-app-a and ewb-bastion listed as running | Pass |
| VPC peering connection | ewb-peering-a-b, pcx-02a215ed0a98562de — Active; both VPC IDs and CIDRs shown (Figure 5) | Pass |
| Peering routes | ewb-rt-a-public: 10.20.0.0/16 → pcx; ewb-rt-a-private: 10.20.0.0/16 → pcx; ewb-rt-b-private: 10.10.0.0/16 → pcx; all Active (Figures 8–10). CLI output shows the same three routes (Figure 12) | Pass — forward and return routes confirmed in all three tables |
| Security groups | ewb-sg-app-a has two SSH (TCP 22) rules and one All ICMP IPv4 rule (Figure 14) | Partial — Source column not visible; the ICMP rule is All ICMP rather than echo request only. Rules for ewb-sg-bastion and ewb-sg-app-b not shown. |
| Public IPv4 addresses | ewb-bastion and ewb-app-a have a public IPv4 address; ewb-app-b has none (Figure 7) | Deviation — the design gives only the bastion a public IP (see Section 7). |
| SSH via bastion | SSH to ewb-bastion, then to ewb-app-b (hostname -I shows 10.20.1.10) (Figure 15) | Pass |
| Private connectivity: App-B → App-A | ping -c 4 10.10.2.10: 4 transmitted, 4 received, 0% loss (Figures 15 and 16) | Pass |
| Private connectivity: App-A → App-B | ssh ewb-app-a, hostname -I shows 10.10.2.10; ping -c 4 10.20.1.10: 4 transmitted, 4 received, 0% loss (Figure 16) | Pass — both directions verified |
| Baseline and controlled-failure tests | Not captured | Not evidenced — not claimed in this report |

# 6. Screenshot Evidence

Evidence is shown in build order: subnets, instances and peering, routes, then connectivity. Account identifiers and public IP addresses have been redacted. Figures captioned as photographs were taken of the screen on 24 October 2026 and are cropped to remove the browser address bar.

## 6.1 VPCs and subnets

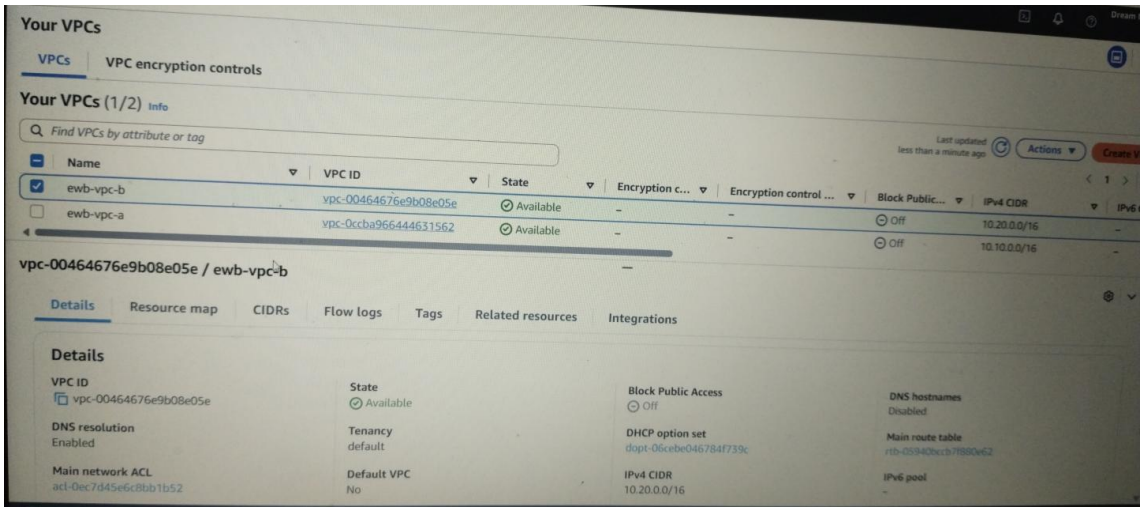



Figure 1 — Your VPCs with ewb-vpc-b selected: ewb-vpc-b (vpc-00464676e9b08e05e) has IPv4 CIDR 10.20.0.0/16 and ewb-vpc-a (vpc-0ccba966444631562) has 10.10.0.0/16; both are Available. Photograph of the console screen, 24 October 2026.

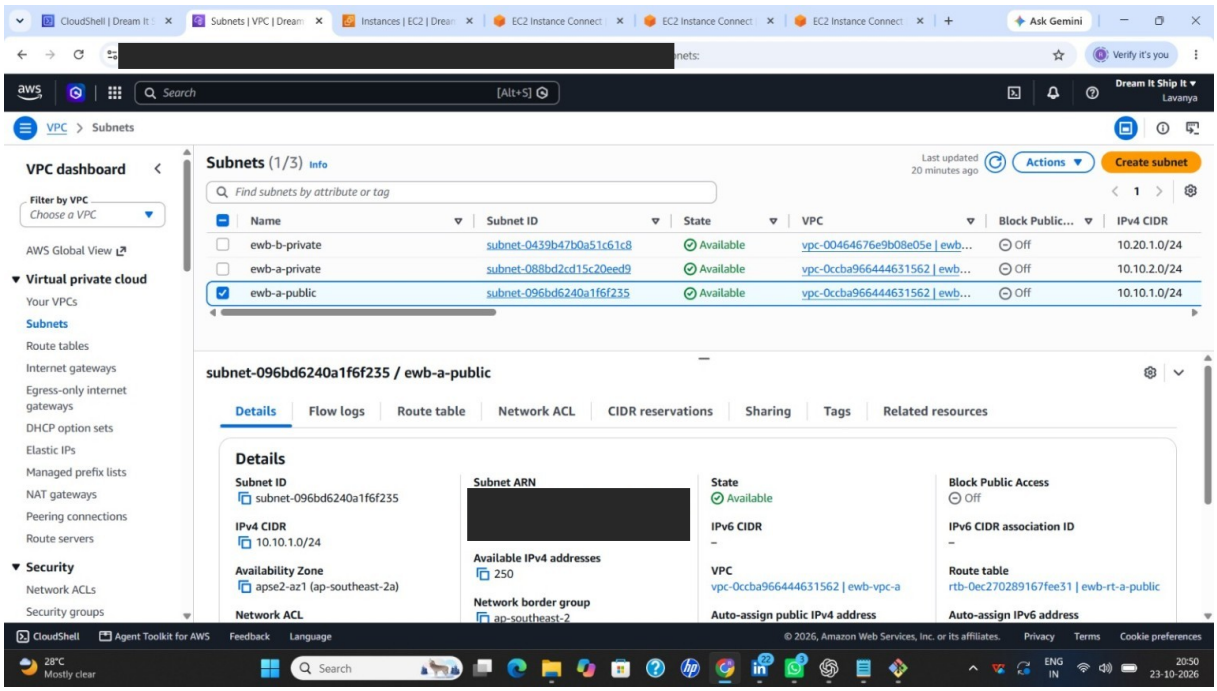



Figure 2 — Subnet list with ewb-a-public selected: 10.10.1.0/24 in VPC A, Available, route table ewb-rt-a-public.

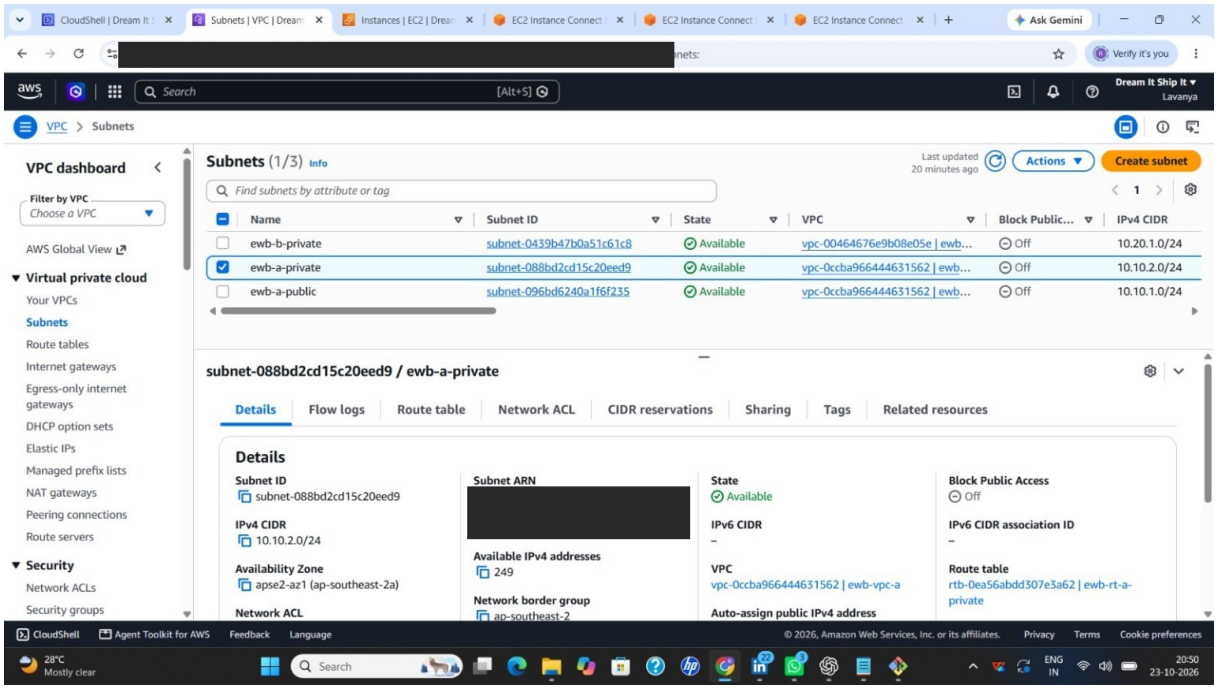



Figure 3 — ewb-a-private selected: 10.10.2.0/24 in VPC A, Available, route table ewb-rt-a-private.

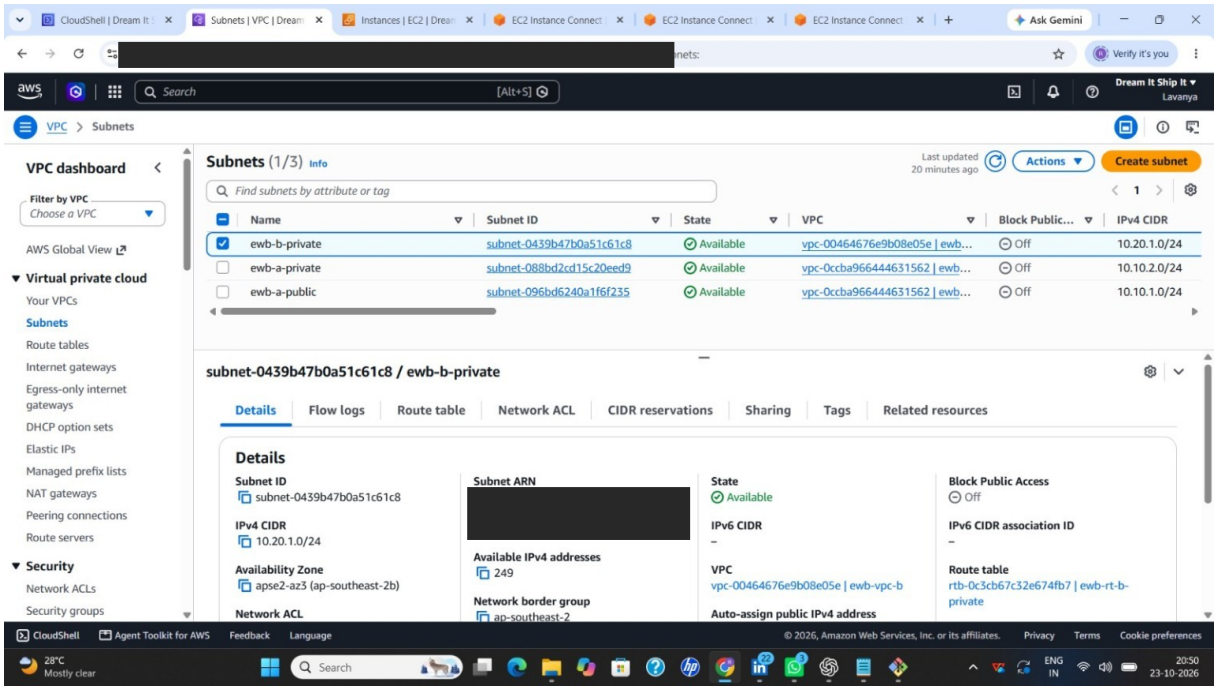



Figure 4 — ewb-b-private selected: 10.20.1.0/24 in VPC B, Available, route table ewb-rt-b-private.

## 6.2 VPC peering and EC2 instances

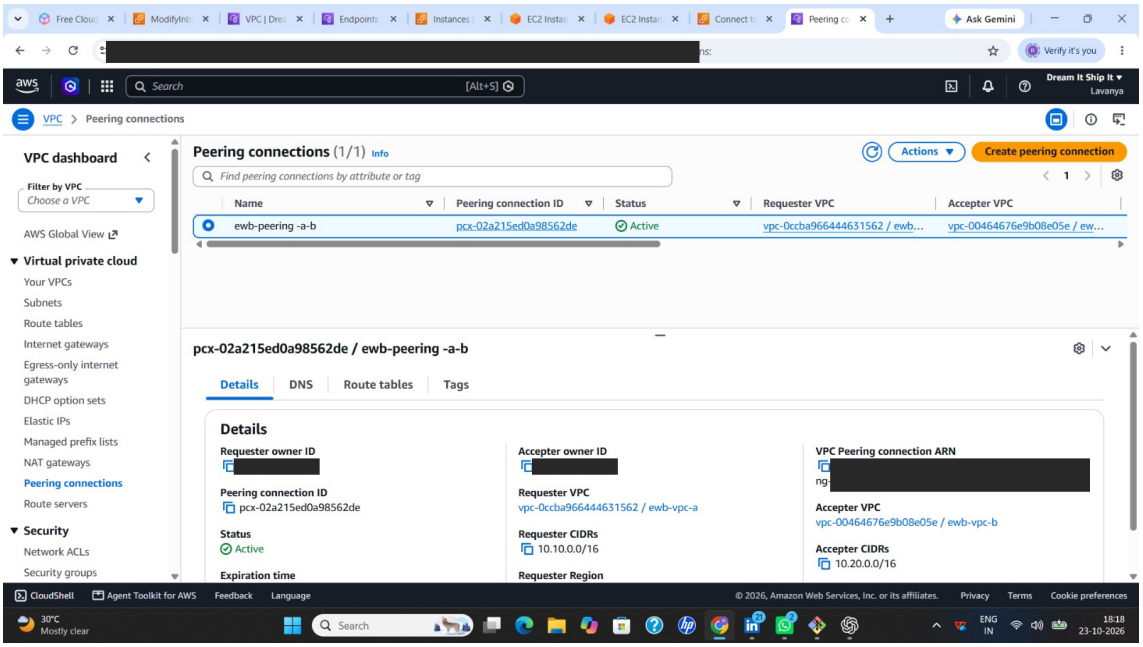



Figure 5 — Peering connections console: ewb-peering-a-b (pcx-02a215ed0a98562de) is Active. Requester ewb-vpc-a (vpc-0ccba966444631562) with CIDR 10.10.0.0/16; accepter ewb-vpc-b (vpc-00464676e9b08e05e) with CIDR 10.20.0.0/16.

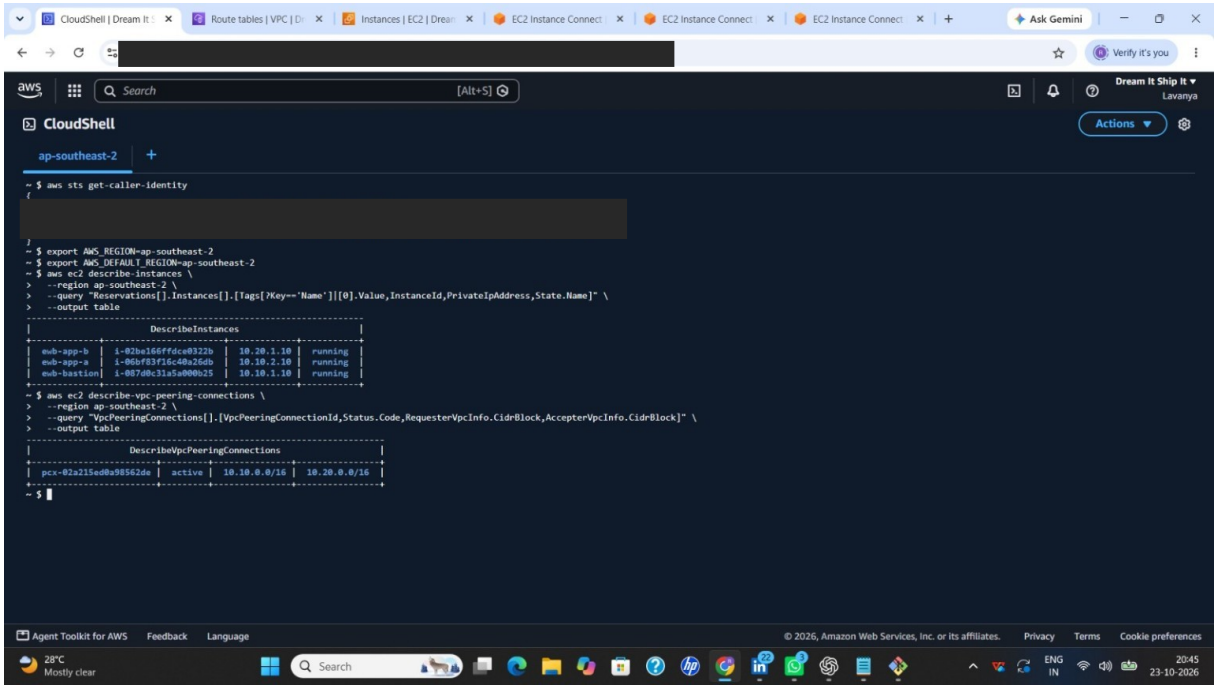



Figure 6 — CloudShell output: ewb-app-b (10.20.1.10), ewb-app-a (10.10.2.10) and ewb-bastion (10.10.1.10) running; peering connection pcx-02a215ed0a98562de is active between 10.10.0.0/16 and 10.20.0.0/16.

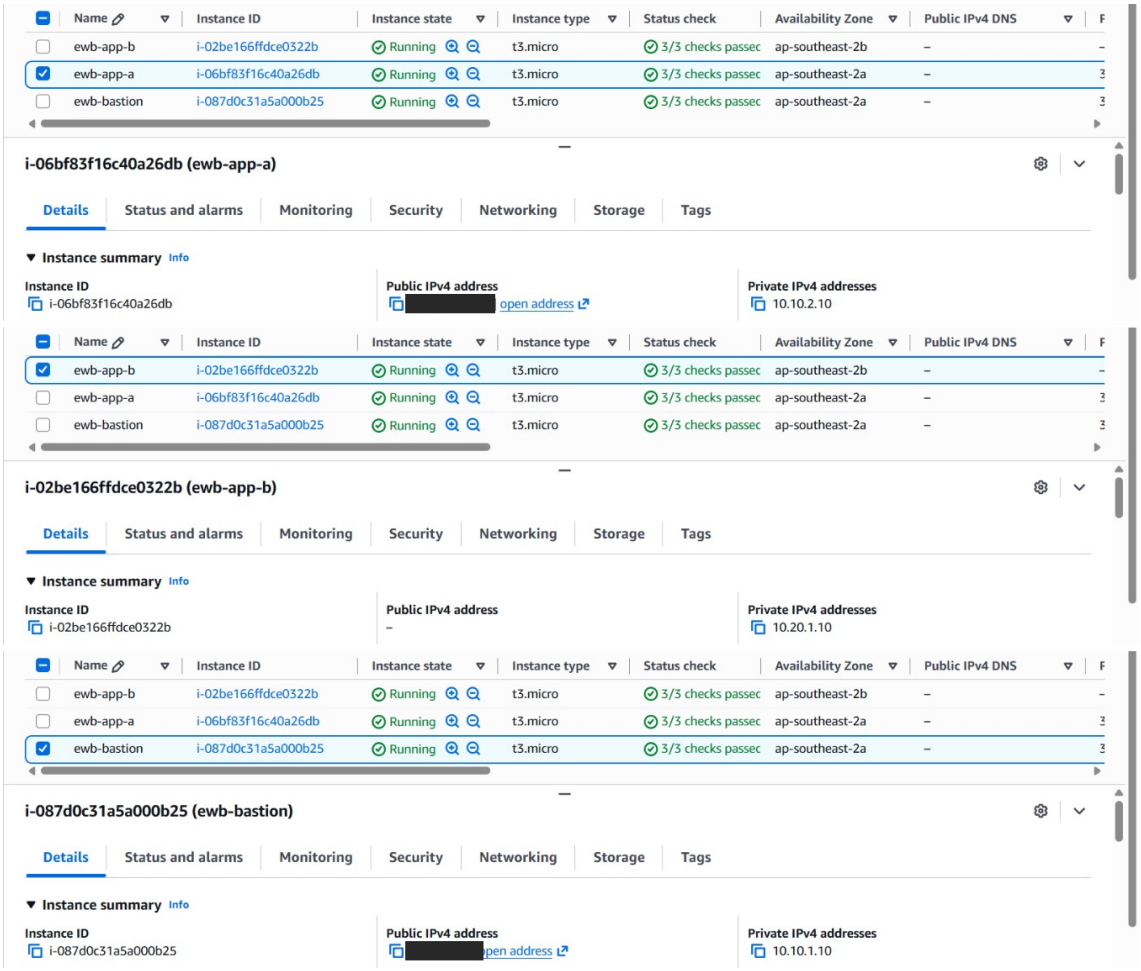



Figure 7 — EC2 console, instance summaries (top to bottom): ewb-app-a (private 10.10.2.10, a public IPv4 address is assigned), ewb-app-b (10.20.1.10, no public IPv4) and ewb-bastion (10.10.1.10, public IPv4 assigned). Public IP values are redacted. All three instances are Running with 3/3 status checks passed.

## 6.3 Route tables

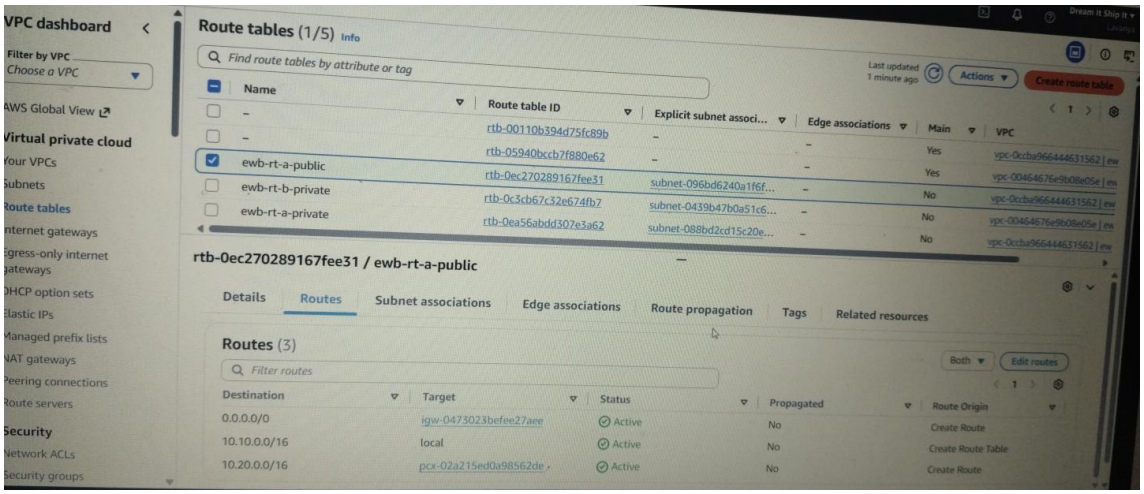



Figure 8 — ewb-rt-a-public (rtb-0ec270289167fee31): 0.0.0.0/0 targets igw-0473023befee27aee, 10.10.0.0/16 is local, and 10.20.0.0/16 targets pcx-02a215ed0a98562de (Active). Photograph of the console screen.

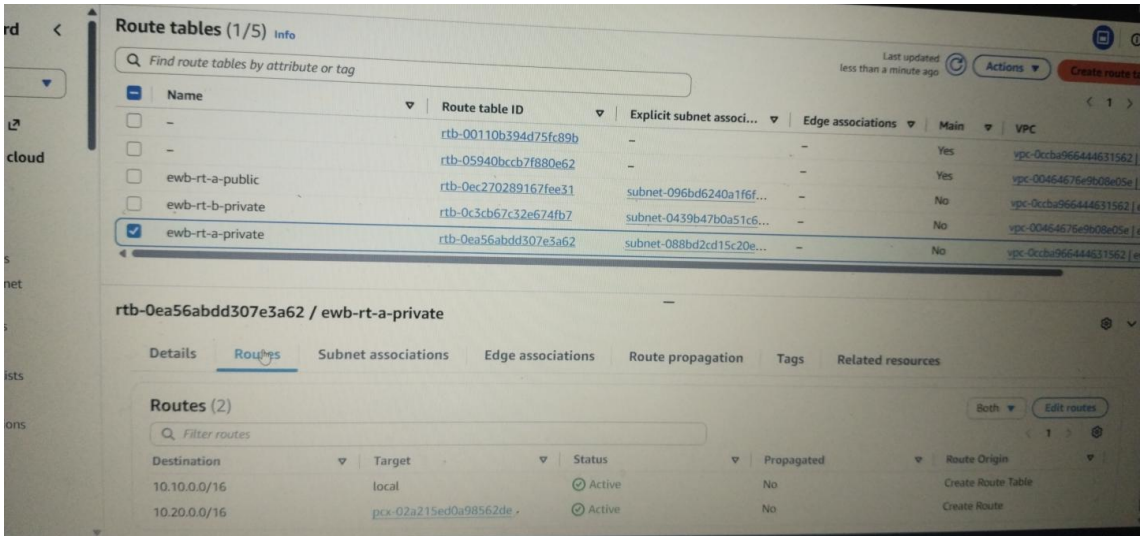



Figure 9 — ewb-rt-a-private (rtb-0ea56abdd307e3a62): 10.10.0.0/16 is local and 10.20.0.0/16 targets pcx-02a215ed0a98562de (Active). There is no internet-gateway route. Photograph of the console screen.

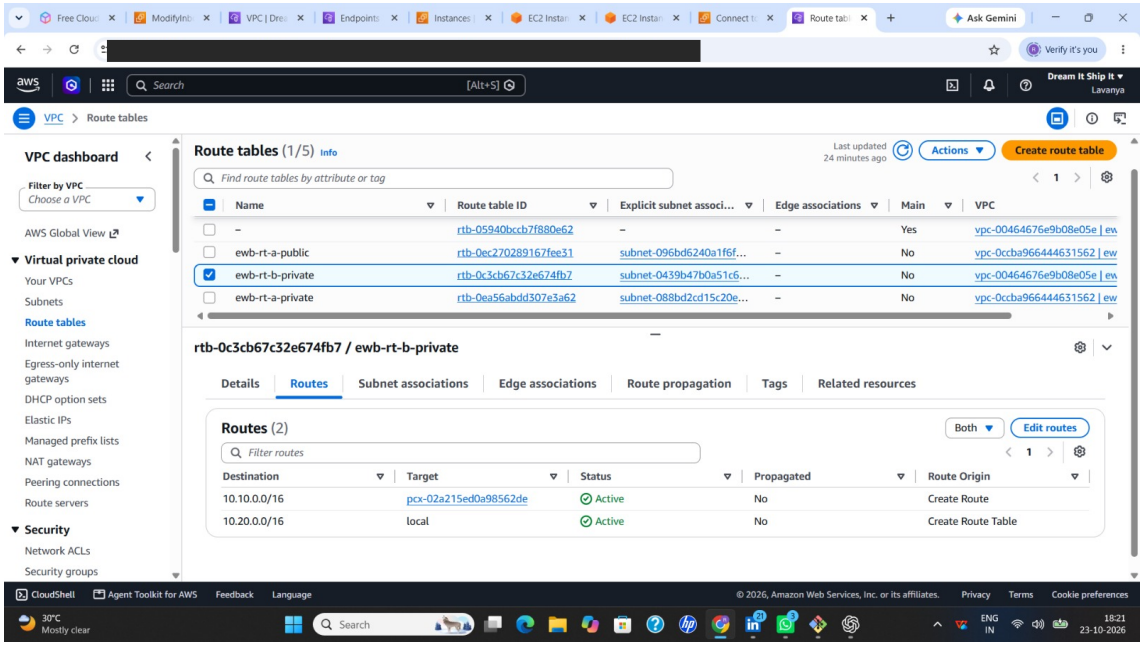



Figure 10 — Route tables console, ewb-rt-b-private (rtb-0c3cb67c32e674fb7, VPC B): 10.10.0.0/16 targets pcx-02a215ed0a98562de (Active) and 10.20.0.0/16 is local. The list also shows each table's ID and explicit subnet association.

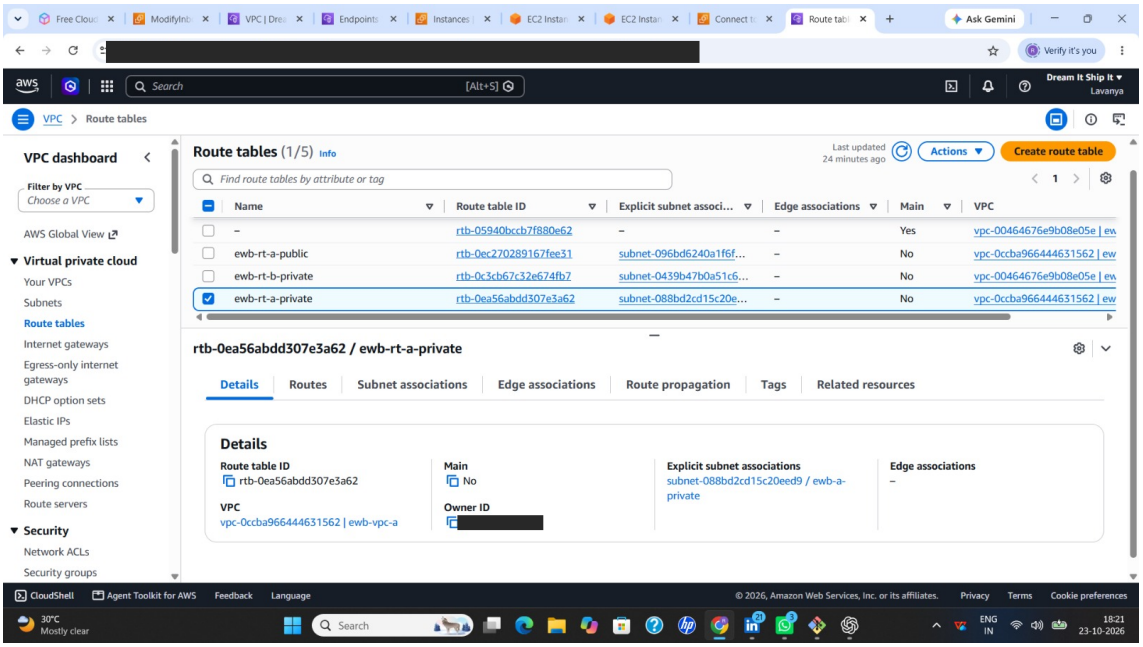



Figure 11 — ewb-rt-a-private (rtb-0ea56abdd307e3a62, VPC A) with explicit association to subnet ewb-a-private (subnet-088bd2cd15c20eed9).

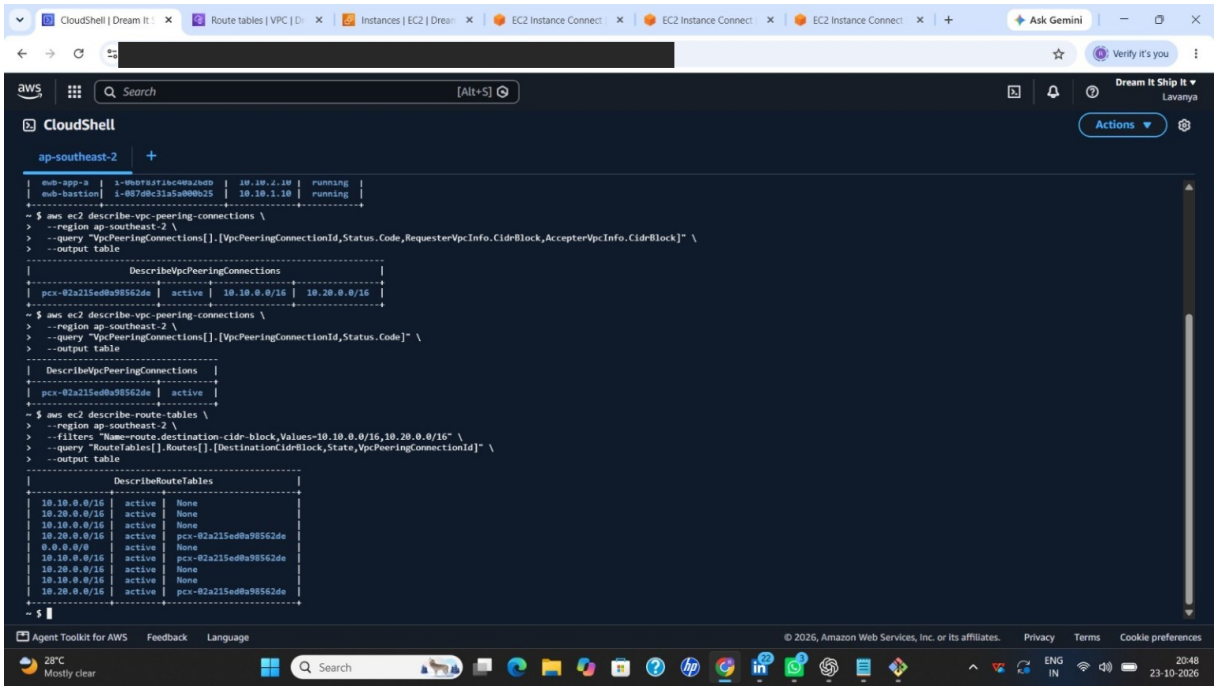



Figure 12 — CloudShell route-table query for destinations 10.10.0.0/16 and 10.20.0.0/16. All routes are active, and three of them target pcx-02a215ed0a98562de.

## 6.4 Security groups

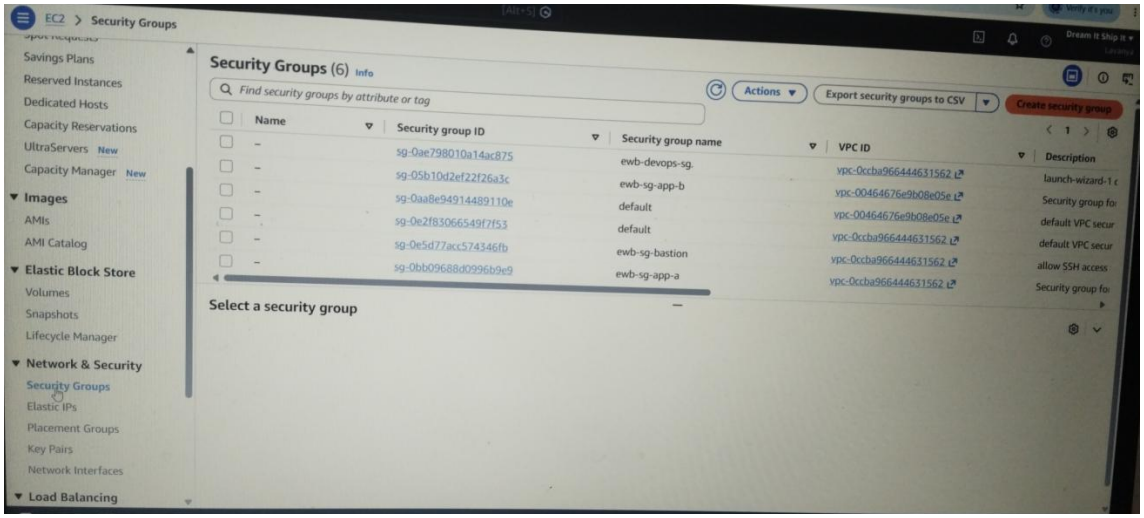



Figure 13 — Security groups list: ewb-sg-bastion, ewb-sg-app-a and ewb-sg-app-b alongside ewb-devops-sg and the two default groups. Photograph of the console screen.

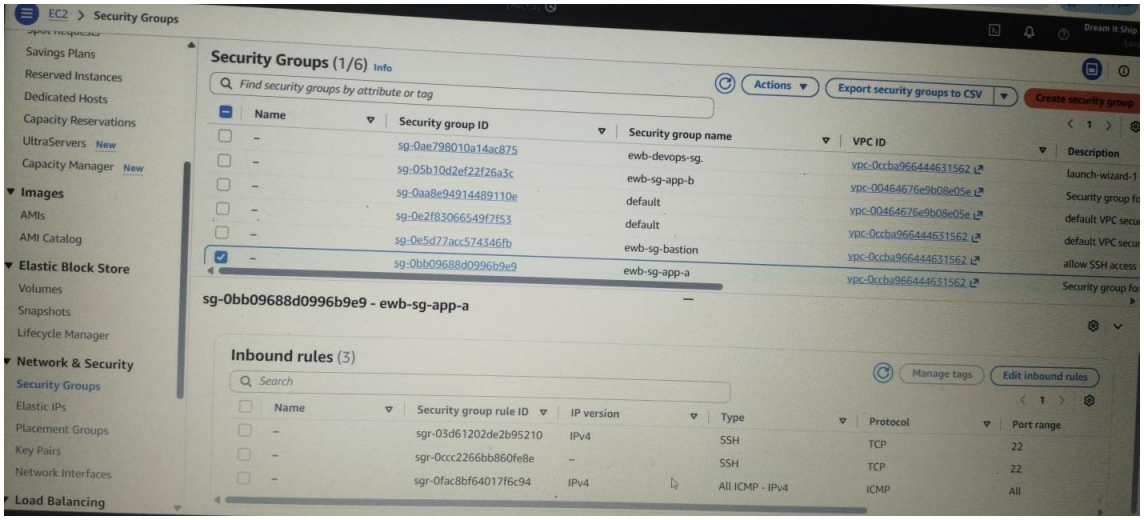



Figure 14 — Inbound rules of ewb-sg-app-a (sg-0bb09688d0996b9e9): two SSH (TCP 22) rules and one All ICMP IPv4 rule. The Source column is not visible in this capture. Photograph of the console screen.

## 6.5 SSH and private connectivity

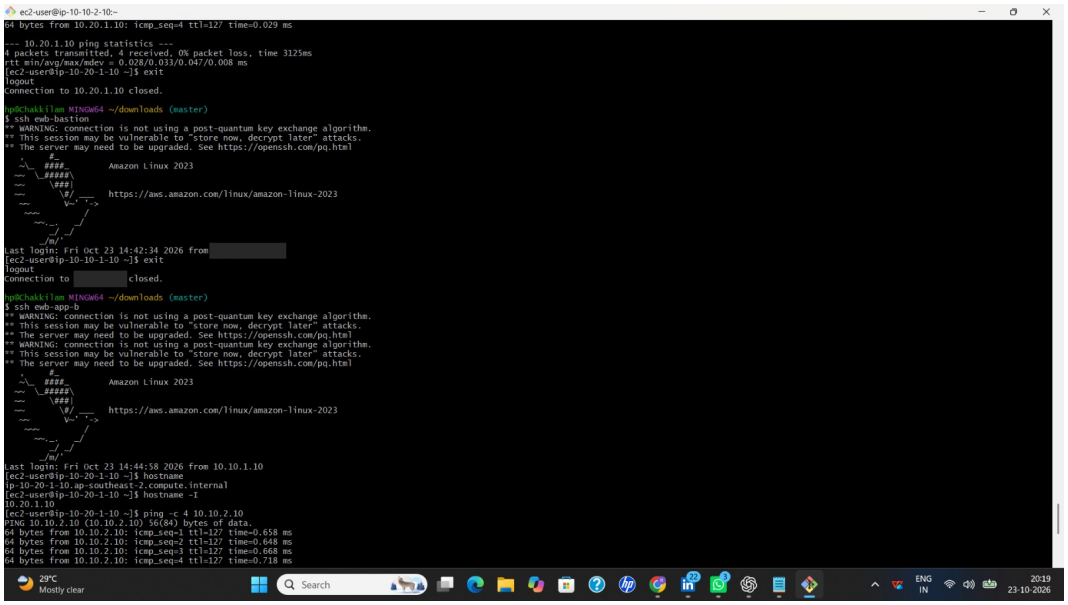



Figure 15 — SSH from the laptop to the bastion and then to ewb-app-b (10.20.1.10), followed by ping -c 4 10.10.2.10 from App-B to App-A (4 replies). The earlier lines show a prior successful ping to 10.20.1.10 (4 received, 0% loss).

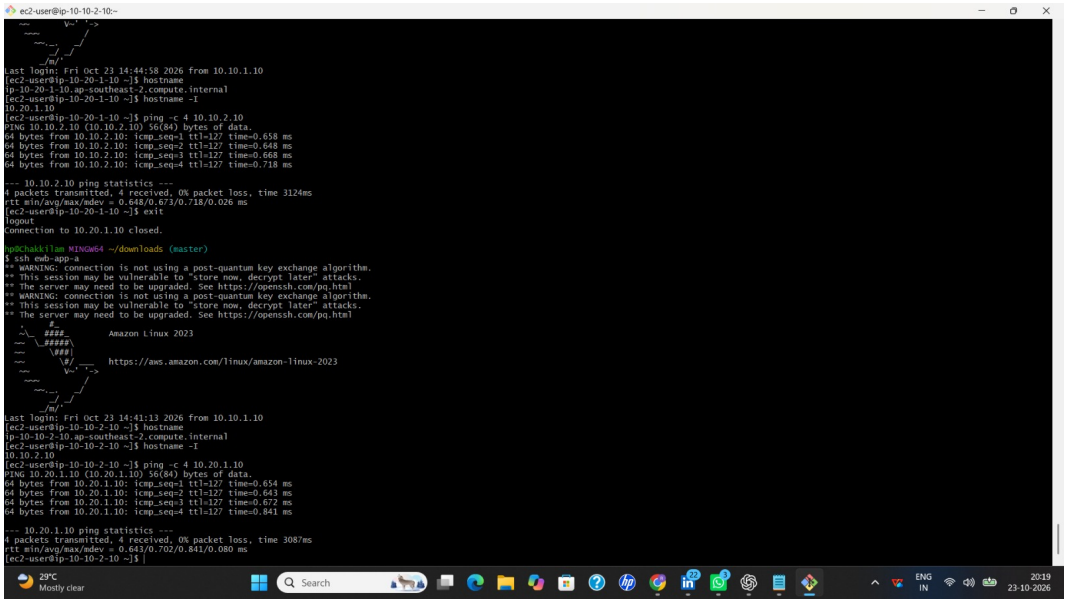



Figure 16 — App-B to App-A ping statistics (4 transmitted, 4 received, 0% packet loss), then SSH to ewb-app-a (10.10.2.10) and the reverse ping to 10.20.1.10: 4 transmitted, 4 received, 0% packet loss.

## 7. Deviations and Items Not Evidenced

The items below were not demonstrated by the evidence in this report, or differ from the target design. They are recorded here so that the report claims only what the screenshots show.

| Item | Note |
|---|---|
| ewb-app-a has a public IPv4 address | Deviation from the design, which gives only the bastion a public IP (Figure 7). ewb-rt-a-private has no internet-gateway route (Figure 9), so the address cannot carry internet traffic, and all access shown in this report goes through the bastion. Recommended action: remove the address. |
| Security-group sources | Rules for ewb-sg-app-a are shown (Figure 14), but the Source column was not captured, and the ICMP rule is All ICMP rather than echo request only. Rules for ewb-sg-bastion and ewb-sg-app-b were not captured. |
| Baseline ping failure before peering routes | Not captured. |
| Controlled ICMP failure and recovery | Not captured. |
| Cleanup | Lab resources were still running at the time of the latest evidence (24 October 2026). Terminate them when finished, as described in Section 8. |

## 8. Security and Cost Controls

- Restrict SSH (TCP 22) on the bastion to your own public IP, never 0.0.0.0/0.
- Allow only the required ports between the private CIDR ranges. Peering does not bypass security groups or network ACLs.
- Keep application instances in private subnets and keep the bastion tightly restricted and patched.
- Never include .pem private keys, passwords, access keys or tokens in screenshots or source control.
- Check EC2, EBS, public IPv4, NAT Gateway and data-transfer charges in AWS Billing. Terminate resources when finished; stopping an instance does not delete its EBS volumes.
- Avoid NAT Gateways and Elastic IPs unless required, as they can add charges.

## 9. Conclusion

The core VPC peering setup is implemented and verified: two non-overlapping VPCs, three subnets with the expected route-table associations, three running EC2 instances, an active peering connection and routes targeting it. SSH access through the bastion and successful pings in both directions (App-B to App-A and App-A to App-B, 0% packet loss) demonstrate private connectivity across the peering connection. The report documents one deviation from the design (the public IPv4 address on ewb-app-a) and lists what was not captured: the security-group Source column, the baseline and controlled failure tests, and cleanup. These are not claimed as done (Section 7).